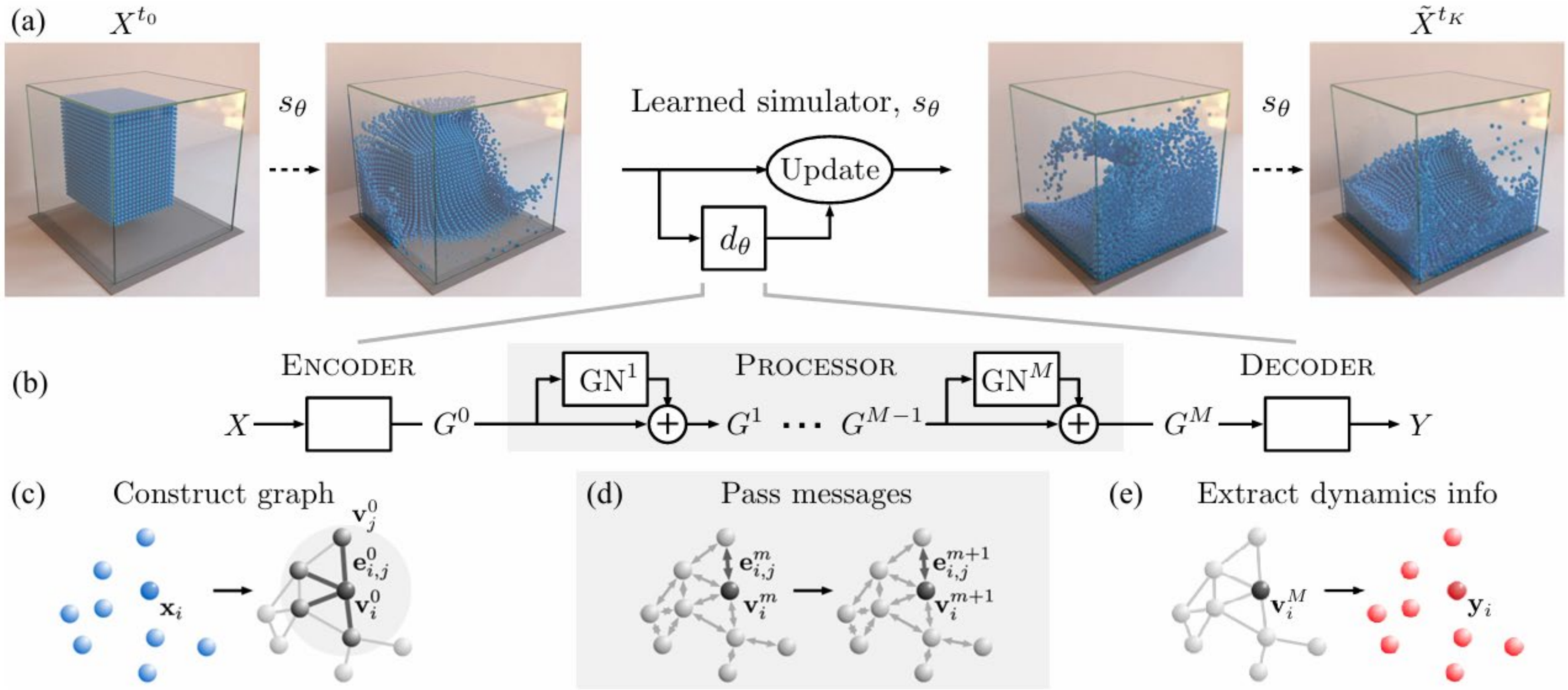
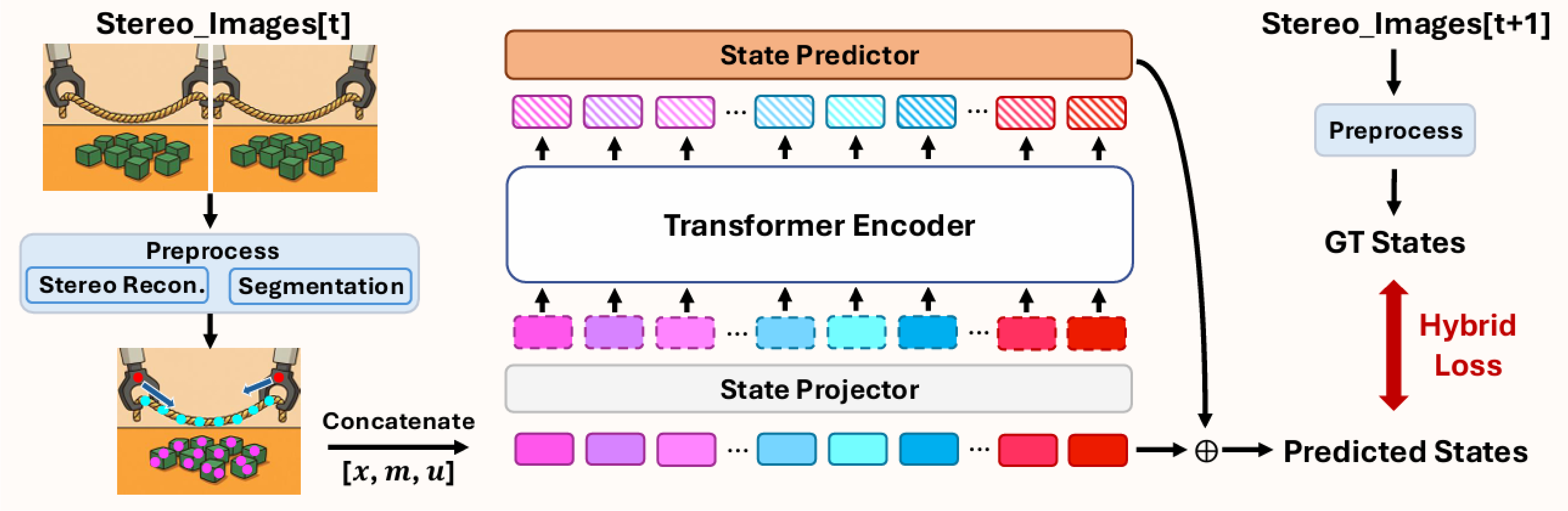
Hacker2

Chengyu Yang



GNN-based methods use multiple layers to learn the interaction laws among the particles but can be sensitive to hyper-parameters, such as the maximum distance for establishing particle adjacency, and the maximum number of neighbors.

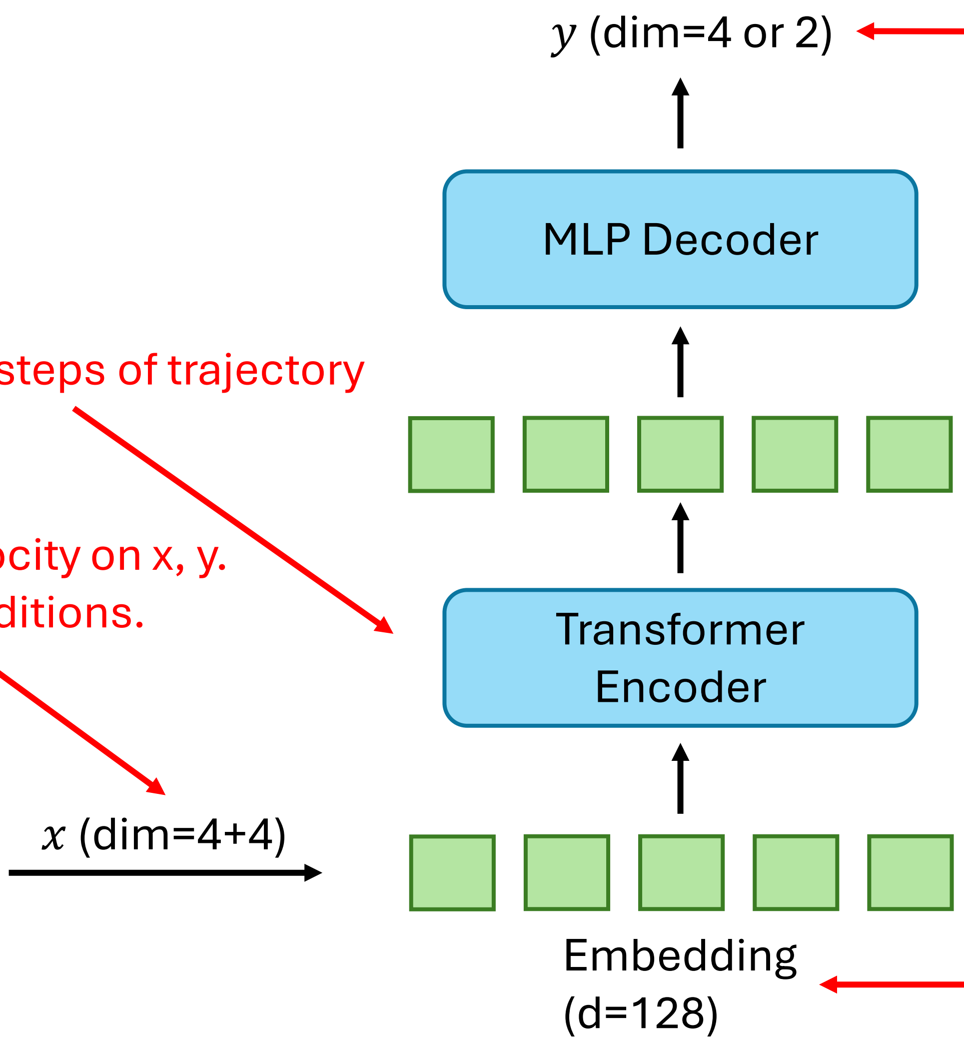


The transformer design allows model to take in all particles simultaneously, thus getting rid of such hyper-parameters of GNN-based methods.

Tried both prediction of
[pos_x, pos_y, vel_x, vel_y]
and [acc_x, acc_y].

8 heads, 15 steps of trajectory

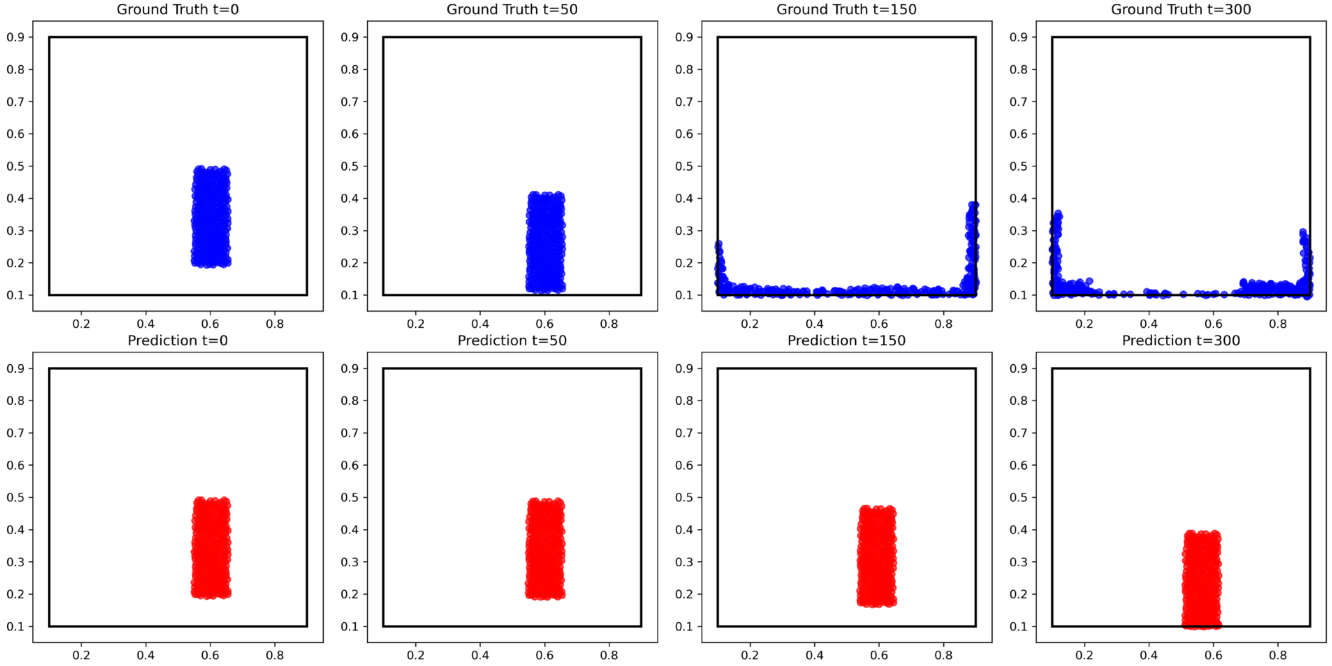
Position & velocity on x, y.
Boundary conditions.



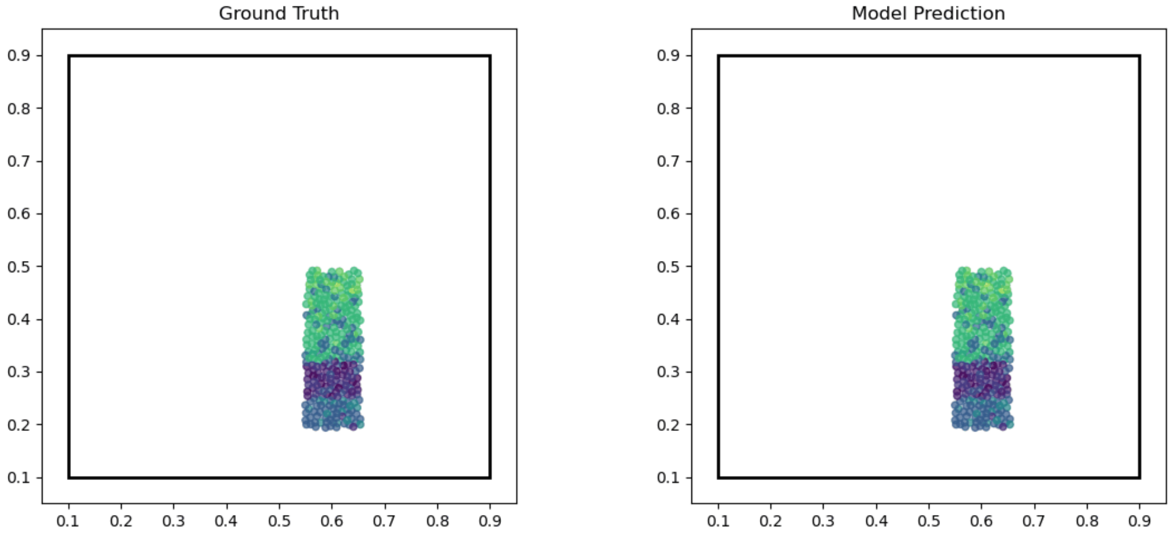
- Applied velocity and acceleration integration in position prediction.
- MSE of position and velocity handles neighbor distance between prediction and ground truth.
- Does not directly penalize large worst-case deviations (Hausdorff Distance), but integration introduces momentum to help.

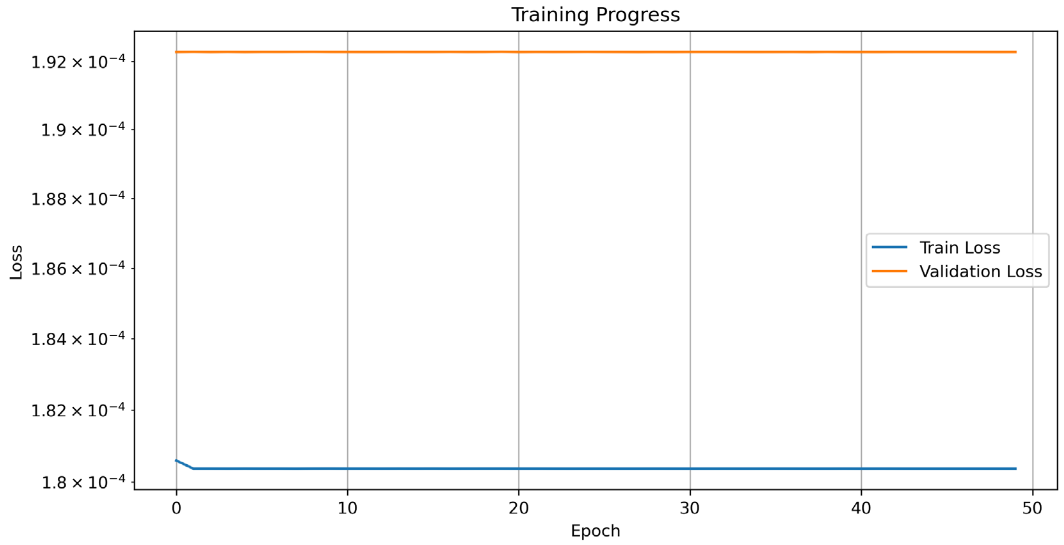
No position embedding needed according to ParticleFormer, but my trial shows better results with position embedding (simple particle indices).

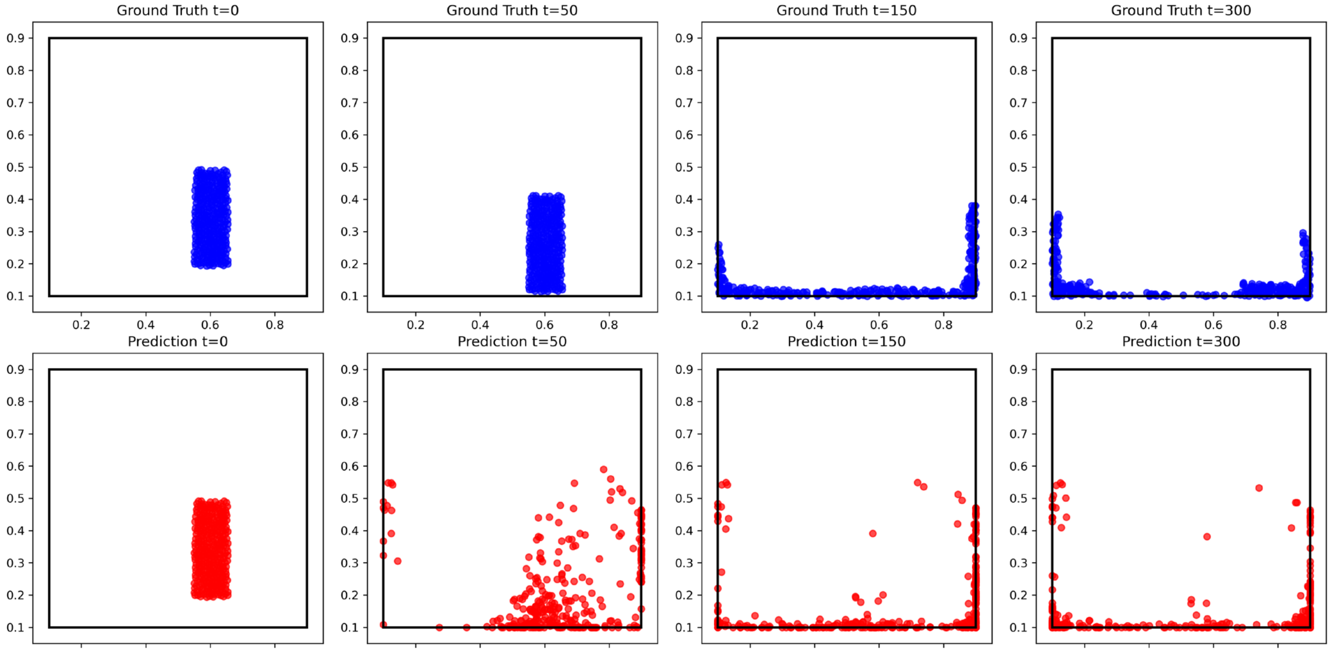
As the code of ParticleFormer has not been published yet, I tried to use a simple transformer model to simulate a water drop procedure.



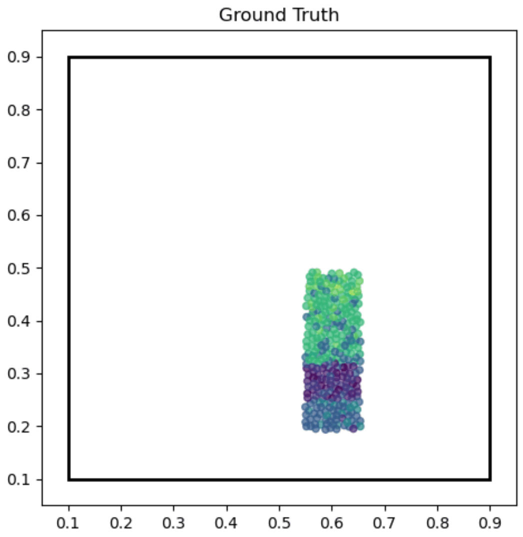
- Output: [acc_x, acc_y]
- Good integrity of shape during dropping but does not deform after hitting the ground.
- It takes time to integrate acceleration, so the initial error is too small.

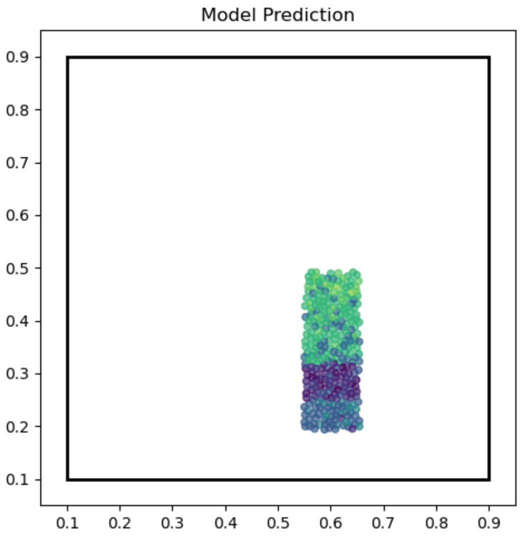


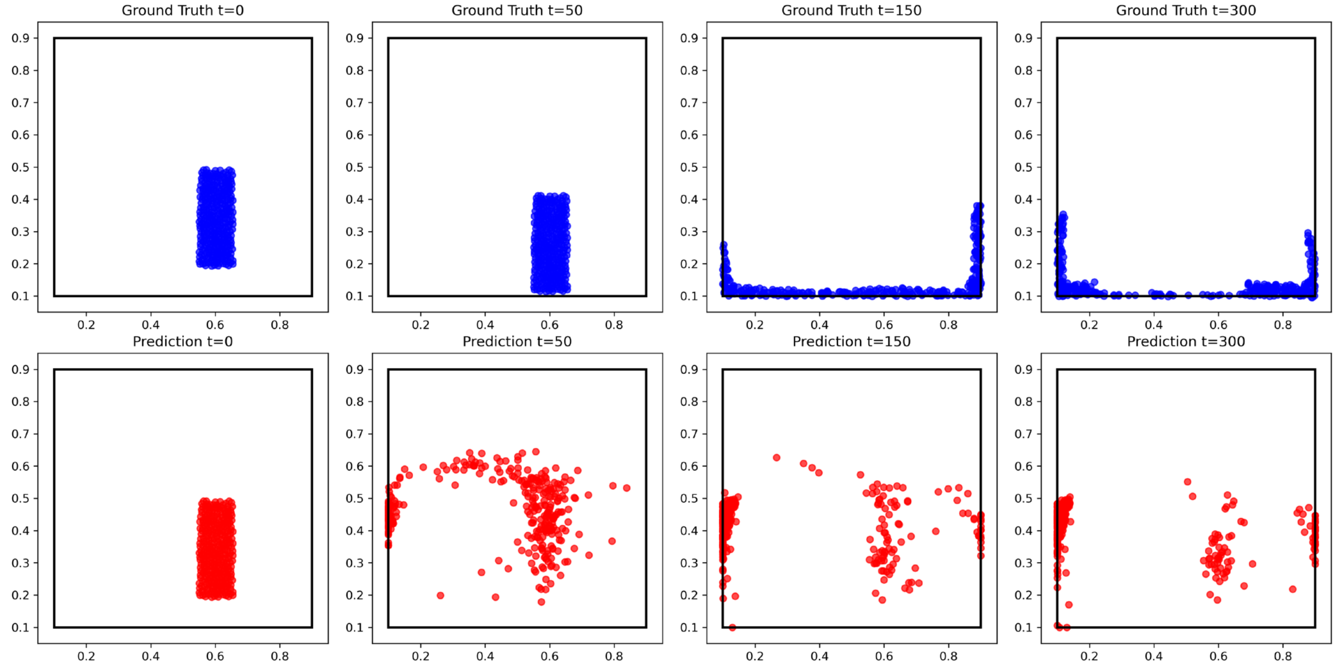




- Output: [pos_x, pos_y, vel_x, vel_y]
- No velocity integration.
- Good fitting in the end but cannot maintain shape integrity during dropping.







- Output: [pos_x, pos_y, vel_x, vel_y]
- With velocity integration.
- A very interesting mixture of the above two cases.